

PRACTICAL 3 (DAA)

Max Heap Sort and Min Heap Sort

Aim: To implement and compare Max Heap Sort and Min Heap Sort using C++.

Key Concepts

- Max Heap: Parent node is greater than or equal to its children.
- Min Heap: Parent node is smaller than or equal to its children.
- Heap Sort time complexity: $O(n \log n)$.
- Heapify is used to maintain the heap property.

C++ Program

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <cstdlib>
#include <ctime>
#include <chrono>

using namespace std;
using namespace chrono;

// =====
// MAX HEAPIFY
// =====
// A Max Heap means:
// Parent >= Children
//
// This function fixes the heap starting from index i.
// =====
void maxHeapify(vector<int>& arr, int n, int i)
{
    int largest = i;          // Assume parent is largest

    int left = 2 * i + 1;     // Left child index
    int right = 2 * i + 2;    // Right child index

    // Check if left child is greater than parent
    if (left < n && arr[left] > arr[largest])
    {
        largest = left;
    }

    // Check if right child is greater
    if (right < n && arr[right] > arr[largest])
    {
        largest = right;
    }

    // If child is greater than parent
    if (largest != i)
    {
        swap(arr[i], arr[largest]);

        // Fix the affected subtree
        maxHeapify(arr, n, largest);
    }
}

// =====
// MAX HEAP SORT
// =====
void maxHeapSort(vector<int>& arr)
{
    int n = arr.size();

    // STEP 1: Create a Max Heap
    for (int i = n / 2 - 1; i >= 0; i--)
    {
```

```

        maxHeapify(arr, n, i);
    }

    // STEP 2: Move largest element to the end
    for (int i = n - 1; i > 0; i--)
    {
        swap(arr[0], arr[i]);

        // Heap size becomes smaller
        maxHeapify(arr, i, 0);
    }
}

// =====
// MIN HEAPIFY
// =====
// A Min Heap means:
// Parent <= Children
// =====
void minHeapify(vector<int>& arr, int n, int i)
{
    int smallest = i;        // Assume parent is smallest

    int left = 2 * i + 1;    // Left child
    int right = 2 * i + 2;   // Right child

    // Check if left child is smaller
    if (left < n && arr[left] < arr[smallest])
    {
        smallest = left;
    }

    // Check if right child is smaller
    if (right < n && arr[right] < arr[smallest])
    {
        smallest = right;
    }

    // If child is smaller than parent
    if (smallest != i)
    {
        swap(arr[i], arr[smallest]);

        // Fix the affected subtree
        minHeapify(arr, n, smallest);
    }
}

// =====
// MIN HEAP SORT
// =====
void minHeapSort(vector<int>& arr)
{
    int n = arr.size();

    // STEP 1: Create a Min Heap
    for (int i = n / 2 - 1; i >= 0; i--)
    {
        minHeapify(arr, n, i);
    }

    // STEP 2: Move smallest element to the end
    for (int i = n - 1; i > 0; i--)
    {
        swap(arr[0], arr[i]);

        // Heap size becomes smaller
        minHeapify(arr, i, 0);
    }

    // Min Heap gives descending order.
    // Reverse it to get ascending order.
    reverse(arr.begin(), arr.end());
}

// =====
// DISPLAY ARRAY
// =====
void displayArray(vector<int>& arr)
{
    for (int i = 0; i < arr.size(); i++)

```

```

    {
        cout << arr[i] << " ";
    }

    cout << endl;
}

// =====
// MAIN FUNCTION
// =====
int main()
{
    int n;

    // Ask user for number of elements
    cout << "Enter number of elements: ";
    cin >> n;

    // Check whether input is valid
    if (n <= 0)
    {
        cout << "Invalid input!";
        return 0;
    }

    // Create an array
    vector<int> original(n);

    // Start random number generator
    srand(time(0));

    // Generate random numbers
    for (int i = 0; i < n; i++)
    {
        original[i] = rand() % 100;
    }

    // Make two copies of the same array
    vector<int> maxArray = original;
    vector<int> minArray = original;

    // =====
    // DISPLAY ORIGINAL ARRAY
    // =====

    cout << "\nOriginal Array: ";
    displayArray(original);

    // =====
    // MAX HEAP SORT
    // =====

    auto startMax = high_resolution_clock::now();

    maxHeapSort(maxArray);

    auto endMax = high_resolution_clock::now();

    // Calculate time
    auto maxTime =
        duration_cast<nanoseconds>(endMax - startMax);

    // =====
    // MIN HEAP SORT
    // =====

    auto startMin = high_resolution_clock::now();

    minHeapSort(minArray);

    auto endMin = high_resolution_clock::now();

    // Calculate time
    auto minTime =
        duration_cast<nanoseconds>(endMin - startMin);

    // =====
    // DISPLAY MAX HEAP RESULT
    // =====

    cout << "\n-----";

```

```

cout << "\n      MAX HEAP SORT";
cout << "\n-----";

cout << "\nSorted Array: ";
displayArray(maxArray);

cout << "Time: "
    << maxTime.count()
    << " nanoseconds";

// =====
// DISPLAY MIN HEAP RESULT
// =====

cout << "\n\n-----";
cout << "\n      MIN HEAP SORT";
cout << "\n-----";

cout << "\nSorted Array: ";
displayArray(minArray);

cout << "Time: "
    << minTime.count()
    << " nanoseconds";

cout << "\n";

return 0;
}

```